第1章　绪论

- ✓C、C++、VC++的关系
- ✓封装、继承、多态的概念及其在C++面向对象编程中的实现

第2章　C++对C的扩展

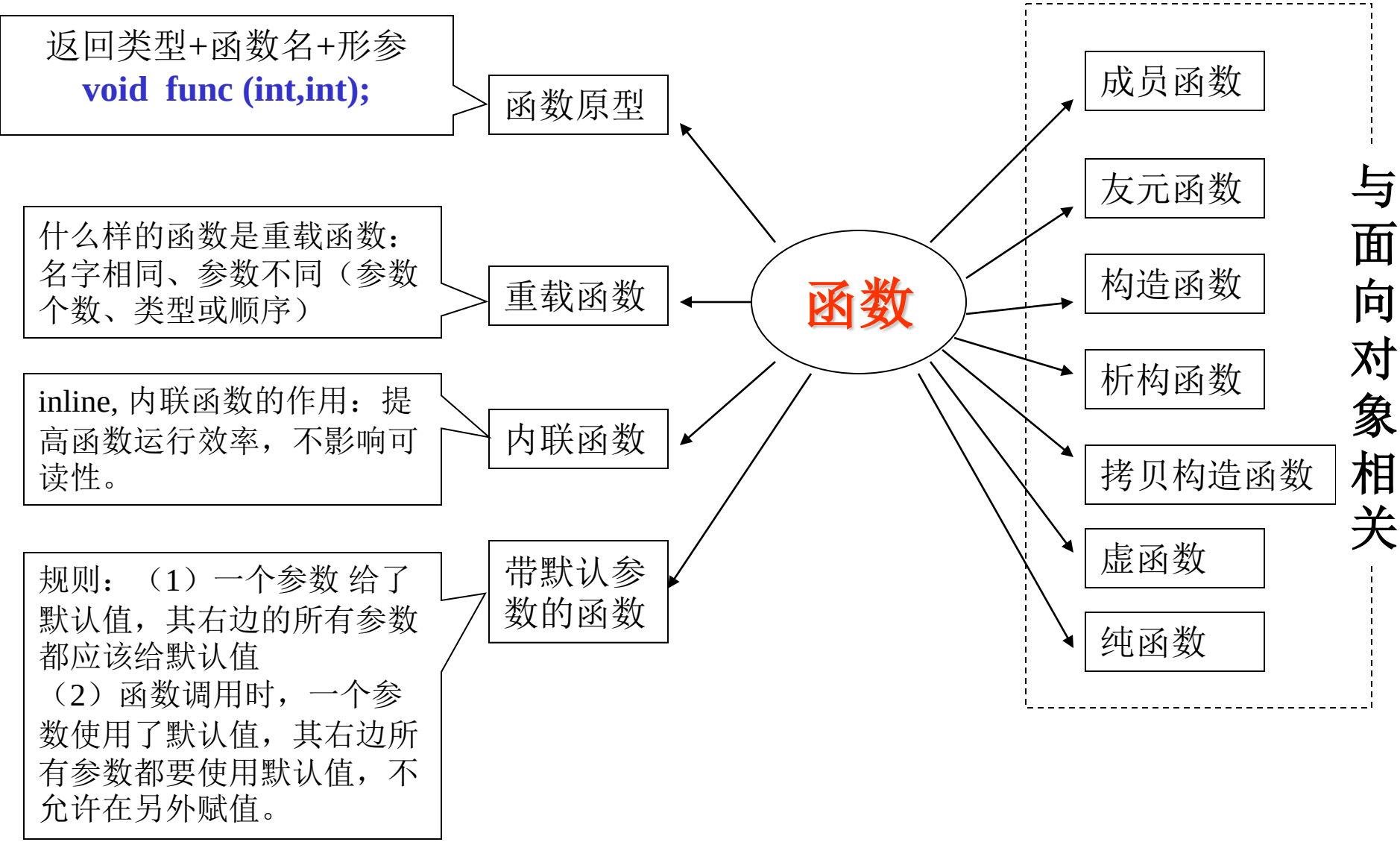
- ✓指针和引用：经常作为函数的形参在面向对象编程中使用，熟练掌握指针和引用的实质是关键。
- ✓声明一个引用必须初始化。
- ✓对指针的限定

  指向常量的指针：**const int *p**　或：**int const *p**

  指针常量：**int * const p**（需要初始化）

  指向常量的指针常量：**const int * const p**（需要初始化）
- ✓new 和 delete运算符
- ✓内联函数
- ✓带有默认参数的函数
- ✓重载函数
- ✓C++的基本语法规则

第2章　C++对C的扩展


函数
函数原型
返回类型+函数名+形参
void func (int,int);
重载函数
什么样的函数是重载函数：名字相同、参数不同（参数个数、类型或顺序）
内联函数
inline, 内联函数的作用：提高函数运行效率，不影响可读性。
带默认参数的函数
规则：（1）一个参数 给了默认值，其右边的所有参数都应该给默认值
（2）函数调用时，一个参数使用了默认值，其右边所有参数都要使用默认值，不允许在另外赋值。
成员函数
友元函数
构造函数
析构函数
拷贝构造函数
虚函数
纯函数
与面向对象相关

第3章　类与对象

✓ 类的定义与实现。类和对象的关系

✓ 类的成员函数与数据成员。访问类的成员的基本规则

类内如何访问?

类外如何访问? 对象、对象指针、对象引用访问…

类外定义成员函数的实现必须指明成员函数所属的类:

```
bool Tdate::IsLeapYear()
  { return (year%4==0 &&year%100!=0) ||(year%400==0);}
```

✓ 类的访问权限： **public: | protected: | private:**

✓ 类的数据成员初始化：由类的构造函数完成，不可以直接初始化（赋值）。

```
class Tdate
{ public:  void Set(int m,int d,int y) ;  …
  private:  int month=12 ;…;}; //error
```

✓构造函数、析构函数、拷贝构造函数的定义、作用、使用规则

```
class A
  { A(void);              //构造函数(可以有参数)
    A(const A &a);        //拷贝构造函数
    ~A(void);  };         //析构函数（不可以有参数）
  A & operator = (const A & a); //重载赋值运算符
```

特别注意当类的成员存在指针类型，如字符数组（学生姓名）、字符串等时这四个函数的实现

例：

```
A func(A & x ) {}  // A func(A x ) ?   A & func(A & x ) ?
void main( )
{  A  s, w;   //调用缺省无参构造函数创建对象
   A m=s;   //调用缺省拷贝构造函数创建对象
   A t=func(s); //调用几次拷贝构造函数？
   m=w;   //调用缺省对象赋值函数将对象w的值赋给对象m
   m=func(s);   //先调用拷贝构造函数,再调用对象赋值函数
}  //调用缺省的析构函数依次析构m、w、s
```

✓构造函数与析构函数总结

1.都不需要指定函数返回类型，也不允许有返回值。

2.构造函数可以重载，析构函数不允许重载 。

3.如果对象定义在函数体内，当函数结束时，该对象的析构函数被自动调用。

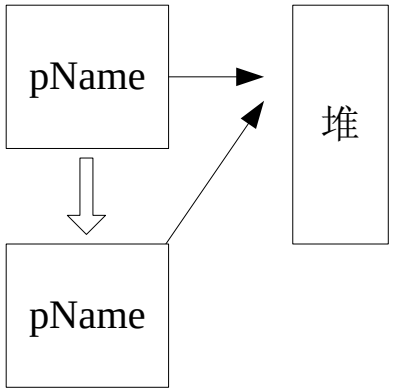
4.当一个对象由new创建，在使用delete时，将会自动调用析构函数

5.当存在继承关系和子对象时，遵循先祖先，后客人，再自己的构造顺序构造类的每一个成员，析构顺序与构造顺序相反。

✓正确理解深拷贝与浅拷贝

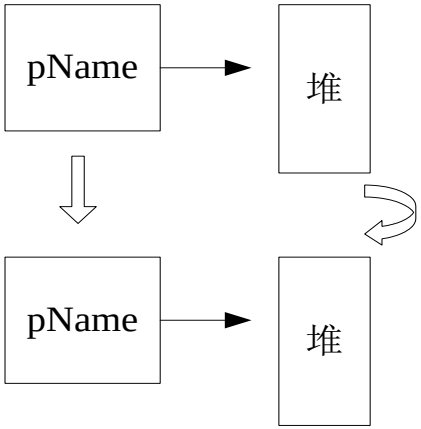
➢如果不主动编写拷贝构造函数，编译器就将以“**浅拷贝**”的方式自动生成缺省的拷贝构造函数。倘若类中有指针变量就隐含错误。

浅拷贝（位拷贝）：只拷贝成员



深拷贝（值拷贝）：

通过自定义拷贝构造函数不仅拷贝成员，还拷贝资源。




```
Person :: Person(Person & p)
{  pName=new char[strlen(p.pName)+1];
   if(pName!=0)
   strcpy(pName,p.pName);}
```

✓对象数组的初始化与对象指针的操作

```
class Point
{ public:
    Point( )
      {x=0;y=0;}
   Point(float a)
      {x=a;y=0;}
   Point(float a, float b)
      {x=a;y=b;}
  Point(Point & p)
      {……;}
};
void main( )
{   Point  array[3]={ Point(3,4), 5 };
    ……}
```

✓对象数组的初始化与对象指针的操作

```
class Counter
{
   private:
     int value;
   public:
    Counter(int v)
     {   value=v;  }
   void show( )
   {
     cout<<value<<endl;
    }
};
```

```
void main( )
{
   Counter array[3]={5, 6, 7};
   Counter *p=array;
                    //或p=&array[0];
   p->show();
   p++;
   p->show();
   p++;
   p->show();
}
```

输出：5
6
7

✓静态成员的作用、初始化、静态成员函数访问非静态成员的规则

```
class Student
{ protected:
    char m_strName[20];
    long m_ID;
    static int m_nCount;  //静态成员变量，表示已创建对象的数量

  public:
    Student(char *strName, long ID); //构造函数
    static int GetCount();           //静态成员函数
    static long GetID(Student);  //对象作为静态成员函数的参数
};
int Student::m_nCount = 0; //在类外初始化静态成员
```

✓友元函数的定义及其使用规则

（1）一个类的友元函数调用时不必加上该类对象名前缀，“**对象.**”。

（2）友元函数说明可以出现在类的任何地方。

（3）在类外实现友元函数时，与普通函数一样，没有必要指明它属于该类,也没有必要用关键字**friend**标识。

（4）在使用友元函数时，必须在参数表中显式地指出它要访问的对象，在函数原型中当然要在形参中指明访问的类。

（5）一个类的成员函数可以作为另一类的友元。

第3章　类与对象

✓类的类成员（子对象）的初始化：参数的传递；冒号语法

```
class StudentID
{ public:
    StudentID(int id)
     { value=id; }
  protected:
     int value;
};
```

```
class Student
{ public:
   Student(char* pName="no name",int ssID=0):id(ssID)
    {  ……… }
  protected:
     char name[20];
     StudentID id;
};

void main()
{ Student s("Randy",9818);
  Student t( “Jenny” );}
```

```
Student(char* pName="no name", StudentID ssID):id(ssID)

void main()
{ StudentID stuid(201401);
s("Randy",stuid); }
```

??

第4章　派生类与继承

✓派生类的定义，三种继承方式下派生类内和派生类的对象访问基类成员的限定

继承方式 基类成员	公有继承 public	保护继承 protected	私有继承 private
公有成员 public	public	protected	private
保护成员 protected	protected	protected	Private
私有成员 private	不可访问	不可访问	不可访问

不同继承方式的影响主要体现在：

- ✓派生类**成员函数**对基类成员的访问权限
- ✓通过派生类**对象**对基类成员的访问权限

✓派生类中如何访问基类的私有成员？

- 将基类的私有成员调整为保护成员。——一般不这样做
- 将派生类或需要使用基类私有成员的成员函数声明为基类的友元

```
class Base
 { private:
    int v1;
    int v2;
  public:
     Base(int a=0,int b=0)
        {  v1=a;v2=b;}
     friend Derived;  };
```

```
class Derived : public Base
   { public: int v3;
       Derived(int a=0)
          { v3=a; }
       void func()
          { int sum1=v1+v2+v3; }};
```

✓派生类与基类同名成员的访问方式

如果在派生类中，或通过派生类对象来使用基类的同名成员，则可以显式地使用作用域限定符，方式如下：

基类名::成员

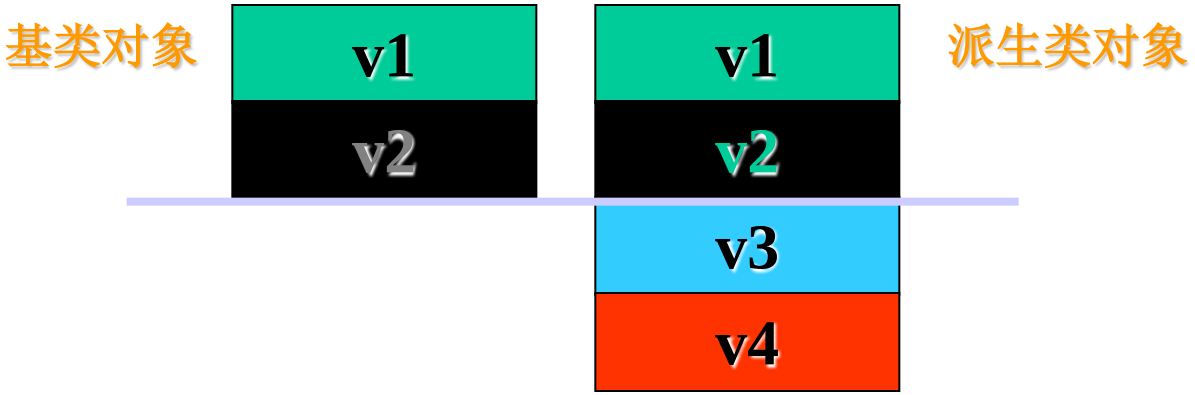
```
class Base
 { protected: int v1;
  public:  int v2;
       Base(int a=0,int b=0)
        {  v1=a;v2=b;}   };

class Derived : public Base
  { int v2;
   public: int v3;
     Derived(int a=0,int b=0)
        { v2=a;v3=b; }
     void func()
       { int sum1=v1+v2+v3;
           int sum2=v1+Base::v2+v3;}};
```

```
void main( )
  { Derived obj(5,6);
    obj.v2=8;  //访问派生类的成员v2 ×
    obj.Base::v2=9; //访问基类的成员v2
  }
```

✓继承与派生模式下对象赋值兼容规则

- 在公有派生方式下，派生类对象可以作为基类对象来使用:
  - 派生类的对象可以直接赋值给基类的对象
  - 基类对象的引用可以引用一个派生类对象
  - 基类对象的指针可以指向一个派生类对象
  - 如果函数的形参是基类对象或基类对象的引用或基类对象的指针，在调用时可以引用或指向派生类对象
- 以上规则不能反过来使用
- 当派生类对象作为基类对象使用时，也只能访问继承自基类的对象



✓ **多继承的基本思想**

理解多继承中的二义性问题。

✓ **虚继承（虚基类）的有关问题**

- 在建立继承关系时，加**virtual**关键字。
- 派生类的对象在构造时对虚基类的成员只构造一次——有效解决二义性问题。
- 带参数的虚基类的初始化。——**如果在虚基类中定义有带形参的构造函数，并且没有定义无参构造函数，则整个继承结构中，所有直接或间接派生类都必须在其构造函数成员初始化列表中列出对虚基类构造函数的调用。**
- 当同时存在虚继承关系、子对象时，类的构造函数和析构函数的调用过程。

第5章　多态性

- ✓ 编译时的多态性——静态联编
- ✓ 运行时的多态性——动态联编

静态联编 { 函数重载
　　运算符重载 }

动态联编 ⟶ 虚函数

- 大多数系统预定义的运算符可以重载，但少数的C++运算符不能重载：

  :: 作用域限定符　　?: 条件运算符

  . 通过对象访问对象成员　* 通过指针间接访问变量

  -> 通过指向对象的指针访问对象成员

  [sizeof 求变量所占内存大小（字节）]

- 不是运算符的符号，如“;”等也不能重载。只能重载已有的运算符，不能定义新的运算符。

- 运算符重载的实质是函数重载。

返回类型 operator 运算符号（参数表）

{ 重载定义 }

第5章　多态性

✓ 两种重载方式：

1、重载为类的成员函数
2、重载为友元函数

✓ 不是所有的运算符都能以两种方式重载：

重载[ ]不可以用友元函数。
重载赋值运算符（=）时，必须重载为成员函数

✓ 同一种运算符友元函数方式重载总是比成员函数方式多一个参数

✓ 运算符++和--有两种使用方式：前缀和后缀。 重载后缀时在参数表中增加一个int类型参数来区分，该参数只用于区别前缀和后缀，一般在函数体里不会引用它。

前缀: ++obj

成员函数方式

```
type className::operator++();
```

友元函数方式

```
type operator++(className&);
```

后缀: obj++

成员函数方式

```
type className::operator++( int );
```

友元函数方式

```
type operator++(className&, int);
```

这里的int类型参数叫做占位参数，用来区别前缀和后缀方式，一般在函数体里不会引用它。

第5章　多态性

✓ 熟悉几个常用运算符的重载方法：

+　　=　　++　　--　　[]

✓ 类型转换函数——类类型转换为其他类型

className::operator type()

{ return type类型的实例；}

- 功能：将className类型的对象转换为type类型的实例。type可以是基本类型，也可以是用户自定义类型。
- 转换函数没有参数，没有返回类型，但函数体内必须有一条返回type类型实例的返回语句。
- 只能将其定义为类的成员函数。

```cpp
class counter
{ int value;
  public:
    //构造函数
    counter(int v=0)
    { value=v; }

    //重载赋值运算符
    counter operator=(int num)
    { value=num;  return *this;

    //类型转换函数
    operator int( )
    { return value; }
};
```

```cpp
void main( )
{   counter obj1=6;
    counter obj2;
    counter obj3(6);
    obj2=10;

//以下均使用类型转换函数
    int v1=obj1;
    int v2=(int)obj2;
    int v3=int(obj2);
}
```

✓虚函数的定义、作用、使用规则

```
class Base
{ public:
  virtual void fn()
     {cout <<"In Base class\n"; }
  };
class SubClass :public Base
{ public:
  virtual void fn() // Virtual可以省略
 { cout <<"In SubClass\n"; }
};
void test(Base & b)
  {  b.fn(); }
```

```
 int main()
{
  Base bc;
  SubClass sc;
  cout <<"Calling test(bc)\n";
  test(bc);
  cout <<"Calling test(sc)\n";
  test(sc);
}
```

✓实现动态联编需要满足的条件

1. 要有说明的虚函数。

2. 调用虚函数操作的只能是基类对象的**指针**或者**引用**。如果采用一般类型的标识对象来操作虚函数，则将采用静态联编方式调用虚函数。

3.虚函数的原型必须一致。——不仅仅是函数名

✓虚函数的限制

1. 只有类的成员函数才能说明为虚函数。

2. 静态成员函数不能是虚函数。

3. 内联函数不能是虚函数。

4. 构造函数不能是虚函数。

5. 析构函数可以是虚函数，而且通常声明为虚函数。

✓ 析构函数可以是虚函数，而且通常声明为虚函数。

```
class A
 {  …
    ~A();
    …};

class B:public A
 { …
   ~B();
   …};

A*pObj = new B;
…
delete pObj; //将会调用A类的析构函数
```

```
class A
 { …
   virtual  ~A();
   …};

class B:public A
 {  …
   virtual  ~B();
    …};

A*pObj = new B;
 …
delete pObj; //将会调用B类的析构函数
```

✓抽象类的概念和定义

1. 抽象类又称纯基类。
2. 包含纯虚函数的类为抽象类。
3. 纯虚函数：
4. 接口类：所有成员函数都是纯虚函数的抽象类

class 类名

{ virtual 数据类型 成员函数名（参数表）= 0; };

```
class Shape  //纯基类，抽象类
{ public:   Shape(double x, double y) :xCoord(x), yCoord(y) {…}
            virtual double Area() const =0;   //纯虚函数
  protected:  double xCoord, yCoord; };
```

✓ **抽象类的使用规则**

1. 抽象类不可以实例化。

2. 抽象类只能作为其它类的基类。

3. 不可以声明抽象类的对象，抽象类的对象也不可以作为函数的形参，但可以声明抽象类的指针或引用，该指针或引用只能指向派生类的对象。

```
int main()
{ Shape  s; // error，Shape为抽象类
  Shape *ps; //不能声明抽象类的对象，但可以声明抽象类的指针

  Circle c(2.0,5.0,4.0);
  ps=&c; //抽象类的指针指向派生类
  cout<<ps->Area()<<endl;

  Rectangle t(2.0,4.0,1.0,2.0);
  ps=&t; //抽象类的指针指向派生类
  cout<<ps->Area()<<endl;  }
```

第六章　模板

✓模板是根据**类型参数**来产生函数和类的机制

```
T max(T a,T b)
  { return a>b?a:b;}
```

✓函数模板与模板函数的定义

//函数模板

template <类型形式参数表>

返回类型 **FuctionName**（形参数表）

{　函数体　}

//模板函数

FuctionName（实参表）；

//函数模板

```
template <typename T>
T max(T a,T b)
   { return a>b?a:b;}

void main()
  {  double a = 1.0;
     double b = 2.0, c;
     c = max(a, b); //模板函数
  }
```

✓ **函数模板和模板函数的使用**

（1）当存在多个不同类型的形参时，需要使用带多个不同的类型参数的模板

```
template <class T ，  class S>
T  max(T a, S b)
    {    return  (T)(a > b ? a : b);    }
```

max(100,100.5);的返回值是多少？

（2）返回值的类型是某一类型参数，则该类型参数必须同时用在函数的参数类型上。

```
template <class T >
T  Function(T a)
    {…}
```

✓函数模板和模板函数的使用

（3）不能使用未在类型模板中定义的类型

```
template<class T, class S>
U FunctionA(T a, S b)
    {…}
```

（4）在template尖括号中的所有类型参数必须在函数的参数表中全部用上。

```
template <class T1 ,class T2, class T3>
T1  Function(T1 a,T2 b)   // error 未使用T3
  {…}
```

第六章　模板

✓类模板与模板类的定义

```
//类模板
template < class Type1, class Type2,…>
class 模板类名
 { //….};
```

模板类名〈实际的类型〉 对象名； // 模板类及其对象

```
template <class T>  //类模板声明
class pair // 类名
 {public:
   pair(T first, T second) //类成员函数
    { Value1=first;
      Value2=second;}
   private:
      T value1,value2;  //数据成员
 };
```

生成模板类并创建对象：

```
pair<int> obj1;
```

✓类模板体外定义成员函数

```
template <类型形式参数表> //定义类模板成员函数
返回类型 className <类型名表> ::MemberFuctionName（形参表）
{    //成员函数定义体  }

template <class T>  //类模板声明
class pair // 类模板定义
  {public:
    pair(T first, T second)；  //类成员函数
   private:
       T value1,value2;  //数据成员};

template <class T>
pair<T>::pair(T first, T second) //在类模板外定义类成员函数
     { Value1=first;
       Value2=second;}
```

第七章　异常处理

✓C++的异常处理机制

检查 ⟶ 抛出 ⟶ 捕获 ⟶ 处理

✓三个关键字

try　　throw　　catch

✓捕获和处理的条件是被抛出的异常的类型与catch语句所声明的类型相匹配

✓catch的参数表中可以只给出类型名，也可以带具体参数，用于接收throw抛出的异常值

```
catch( int )  // ok
catch( int a )  // ok
```

✓catch必须紧跟try语句块之后

✓处理异常的语句放在catch块中

第七章 异常处理

```
#include
using na

class Ou
{ public
   Outof
    {i=a
   int indexValue()
    {return i;}
  private:int i;
};

class Array
{ public:
  int &operator[](int i)
   {if (i<0||i>=10)
     throw OutofBounds(i);
    return a[i]; }
  private: int a[10];
};
```

"J:\面向对象程序设计\备课\ch7_slide22\...

```
a[3]  30
Subscript value 100 out of bounds
Press any key to continue
```

```
运算符函数
[3]<<endl;

[100]<<endl;
   }
  catch(OutofBounds error)
   {cout<<"Subscript value "
   <<error.indexValue()
   <<" out of bounds\n";}
    return;
}
```

构造一个包含10个元素的数组，重载运算符[ ]，给数组元素赋值，进行越界检查，如果下标为负或大于10，报告越界错误。

编程题样例

编写一个名为A的类，

它有两个整型数据成员a、b;

带默认参数的构造函数用于初始化A类的对象（成员默认值均为0）；

另有一个成员函数display用于输出A类的成员的值。

主函数中声明一个A类的对象obj（成员值分别为3和5），并通过类成员屏幕输出该对象成员的值。

编程实现以上功能。

```
#include <iostream.h> ·····1'
class A  ·····1'
{
  int a,b;  ·····1'
  public:
  A(int a1=0,b1=0) ·····1'
    {a=a1;b=b1;} ·····1'
  void display() ·····1'
   {cout<< "a=" <<a<< " b=" <<b<<endl;}
};
void main()
  {A obj(3,5); ·····1'
    obj.display();} ·····1'
```

丰收了！

咸阳烟草杯
和谐咸阳数码摄影作品（长武信息办）